Cipher Audit Platform

CMVP Workshop Handout

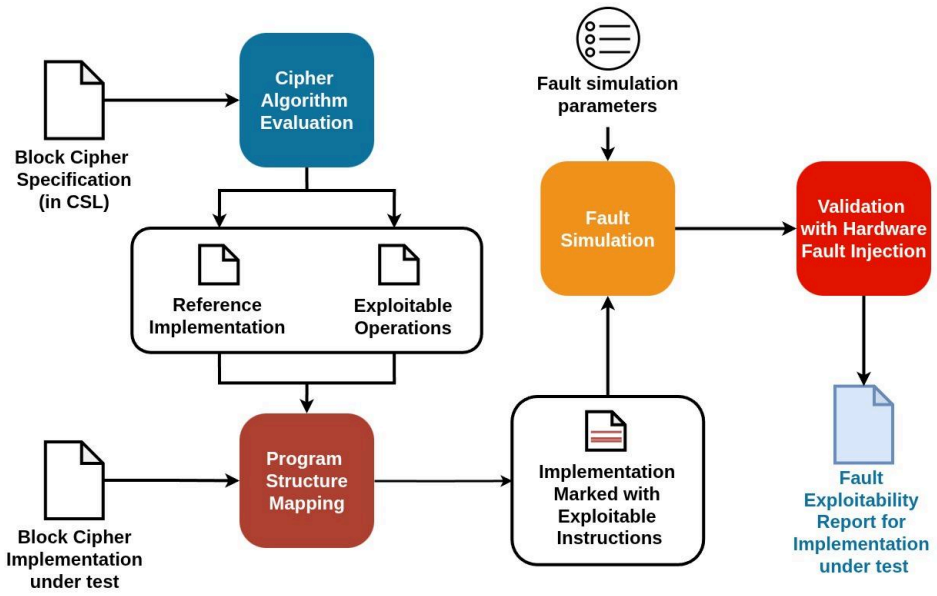
https://github.com/nifey/CipherAuditPlatform

Overview

Cipher Audit Platform (CAP) provides a set of functionality for evaluating block cipher implementations in C, to **quantify their resilience against Differential Fault Analysis Attacks**, and also to **check for zeroization** of sensitive key material after their effective use.

Implementation

Cipher Audit Platform (CAP) is implemented as a Command-line tool, with an optional WebUI, written in Python 3. In addition, the CAP tool integrates the zeroization checks provided by the GNUZero tool.



Cipher Verification Workflow

The above diagram shows the overall planned workflow when using the CAP tool for evaluating resistance against Differential Fault Analysis (DFA) attacks. A cipher implementation along with a Cipher specification should be provided as input. The cipher specification is used to identify round operations vulnerable to DFA attacks, and also for generating a golden reference C implementation. The reference implementation is mapped with the given implementation, and the vulnerable operations in the implementation are identified. The tool also provides a fault simulation module which injects random faults in the program during execution to identify instructions in the binary which are more susceptible to DFA attacks. The output from the tool is the implementation exploitability report that highlights the possible locations which the tool identifies as vulnerable.

The zeroization module (not shown in the diagram) takes an annotated C file as input and outputs error messages if it identifies leakage of sensitive material in memory.

Note: In the current version of the tool, only the golden reference synthesis, fault simulation, and zeroization modules are fully implemented. The other modules are under progress.

01. Cipher Specification Language (CSL)

Cipher Specification Language allows us to precisely describe the rounds of a cipher at a high level, without the implementation details. It is used for generating reference cipher implementations, identifying cipher operations vulnerable to Differential Fault Analysis attacks, and to identify the various cipher rounds in a given implementation.

Syntax and Semantics

```
<begin>
    <declaration>
        SBOX[256] { 63 7c 77 7b ... }
        KEY[16]   { 2b 7e 15 16 ... }
    </declaration>

    <operation>
        <func> < F_MUL2 ( a ) >
            < h : F_RS   ( a , 7 ) >
            < t : F_LS   ( a , 1 ) >
            < n : F_MUL  ( h , '0x1b') >
            < m : F_XOR  ( n , t ) >
            < o : F_AND  ( m , '0xff') >
            < ret o >
        </func>
    </operation>

    < F1 > < nonlinear > < SUBBYTE > <
        < F1[0] : F_LKUP ( F0[0] , SBOX ) >
        < F1[1] : F_LKUP ( F0[1] , SBOX ) >
        < F1[2] : F_LKUP ( F0[2] , SBOX ) >
        < F1[3] : F_LKUP ( F0[3] , SBOX ) >
          ...
    />

<end>
```

Example CSL Specification

Every CSL specification consists of three parts delimited by `<begin>` and `<end>` tags:

1. Declarations
    - Represents the hard-coded data tables used in the Cipher.
      This includes SBox values, T-Tables, and Keys.
    - Delimited by `<declaration>` and `</declaration>` tags.
    - Each declaration consists of a name, length, and data elements as hex values.
    - In the example specification shown above, SBOX and KEY declarations have been defined
2. Operations
    - Represents user-defined operations used in the rounds of the cipher.

- CSL provides a few in-built operations for common arithmetic.
- Delimited by `<operation>` and `</operation>` tags.
- The individual operations are delimited by `<func>` and `</func>` tags. The tag immediately following the `<func>` tag specifies the operation name and the argument list.
- The body of the operation consists of a sequence of operations that operates on the arguments or constants. For example `< h : F_RS ( a , 7 ) >` defines a statement where the variable h will be assigned the value a >> 7. `F_RS` is an in-built function that performs right shift operation.
- The return statement is used to return a computed value. For example `< ret m >`, returns the value held in variable m inside the operation body.
- In the example specification above, a Galois Field multiplication is defined.

3. Cipher Rounds
    - Rounds represent the series of functions that constitute the cipher. In AES, this consists of the SubBytes, ShiftRounds, MixColumns and AddRoundKey operations.
    - Each round specification consists of the round name (which must start with "F" followed by the round number in the sequence of operations), the linearity type of the round (linear/nonlinear), the type of the round (KEYXOR, SUBBYTE, MDS), followed by the *Parts* of the round within `<` and `/>`. The plaintext is represented as F0.
    - Each part of the round consists of a statement that assigns to a Round function value specified by the round name and an index into it.
    - The example specification above shows an SBox round in AES, where a byte from the plaintext F0 is looked up in the SBox.
    - CSL provides a `for` construct to avoid repeatedly writing the same functions for each round in the cipher. CSL also provides a way to perform bit slicing to extract specific bits of a byte. For more details, please refer to the documentation given in the CAP repository.

The CAP tool comes built-in with CSL specifications for standard block ciphers: AES-128, CLEFIA-128, SPECK-128, PRESENT-80, and stream ciphers: ASCON-128, CHACHA20, Trivium, Grain128a.

The current CSL grammar does not model the key expansion step, and so at present the expanded key has to be explicitly specified in the CSL specifications. Future CSL versions will allow the ability to model key expansion.

Installation and Setup

- Install dependencies : Python3, pip and Docker (with the current user able to run docker containers without sudo permissions)
- Download the CipherAuditPlatform source code
  ```
  git clone https://github.com/nifey/CipherAuditPlatform cap
  cd cap
  ```
- Create a python virtual environment, and install dependencies using pip
    - On Linux
      ```
      python -m venv env
      source env/bin/activate
      pip install -e .
      ```
    - On Windows

```
py -m venv env
.\env\Scripts\Activate.ps1
python -m pip install --upgrade pip setuptools wheel
python -m pip install -e .
```
python -m pip install --group dev

- Run CAP
  `python main.py --help`
- CAP tool includes an easy-to-use Web Interface. You can start the Web UI by running the serve command. When serving, open http://127.0.0.1:5000 in your web browser. (Note that the web interface is designed to be used locally, and should not be exposed to the network or the internet.)
  `python main.py serve`
- The zeroization module requires downloading a docker image, which will be automatically downloaded on the first run of the zeroization check. Just ensure that docker works correctly, and the current user has access to docker.

Exercises

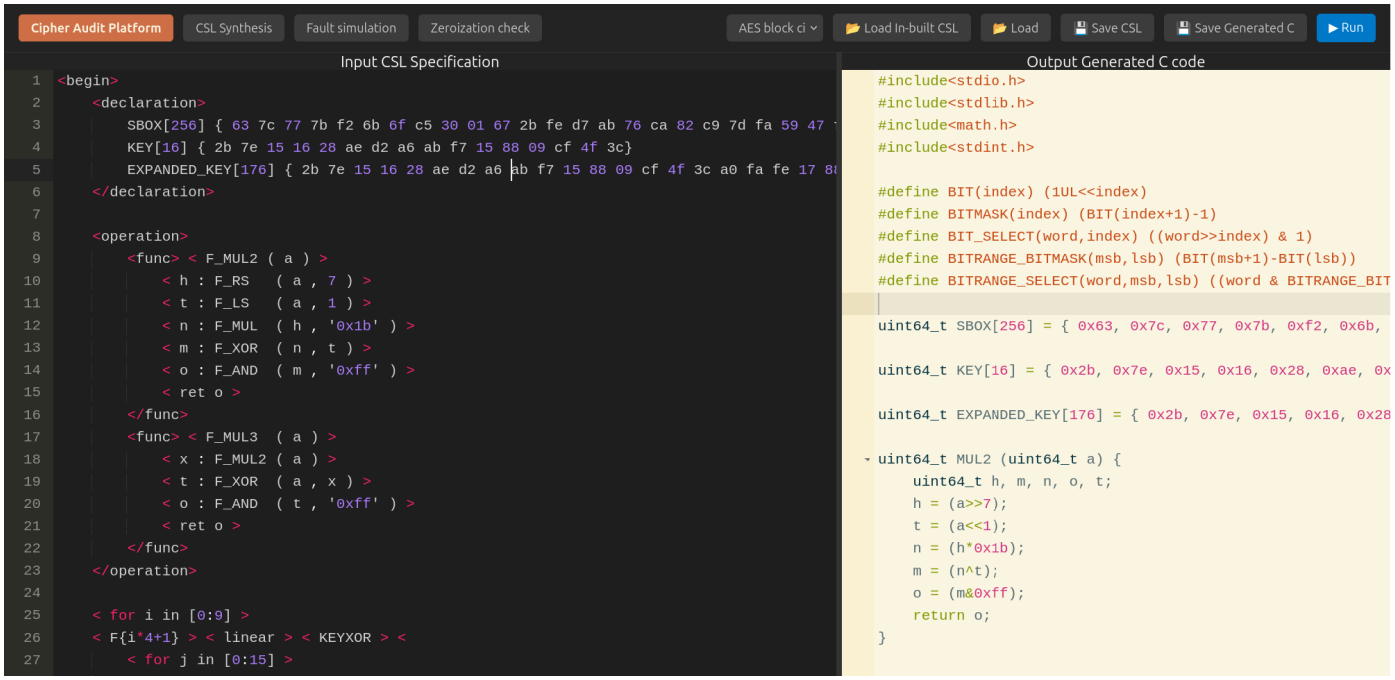
- Follow the installation instructions above to install CAP on your system.
- Run the `serve` command to open and interact with the web interface.

02. Golden reference synthesis

The golden reference synthesis module takes a Cipher specification written in CSL as input and generates the equivalent C code.

Web Interface

- On the web interface, the Golden reference generation is present in the “CSL Synthesis” tab. The left editor pane takes a CSL specification, which would be synthesized on pressing the Run button.
- The dropdown list on the header shows the list of available in-built CSL specifications available. You select one of them, and press `Load in-built CSL` button to load it to the editor. Alternatively, you can load a file from your system using the `Load` button.



CLI Interface

- List the in-built CSL specifications
  `python3 main.py list-csl`
- Show the full CSL specification of any in-built CSL specification
  `python3 main.py list-csl -s <spec-name>`
- Generate the C file for the given CSL specification
  `python3 main.py synthesize-csl <cslfile or in-built spec-name>`

Exercises

- Load and examine the in-built CSL specification of AES block cipher (using either the CLI or Web interface). Try to identify the various rounds that perform SubBytes, ShiftRows, MixColumns, AddRoundKeys operations.
- Generate the reference C implementation using CAP, execute the generated code and examine its output.

03. Fault Injection Simulation

The fault injection simulation module simulates fault in a cipher program within an annotated region within the cipher code, and outputs statistics on how many faults were successful for a Differential Fault Attack on the Cipher. Currently, the tool supports Instruction corruption faults based on a N-bit flip model. The tool iterates through every byte in the specified regions and simulates faults in each of those bytes. The logs are stored in the work directory specified, and can then be used to view statistics about the simulations

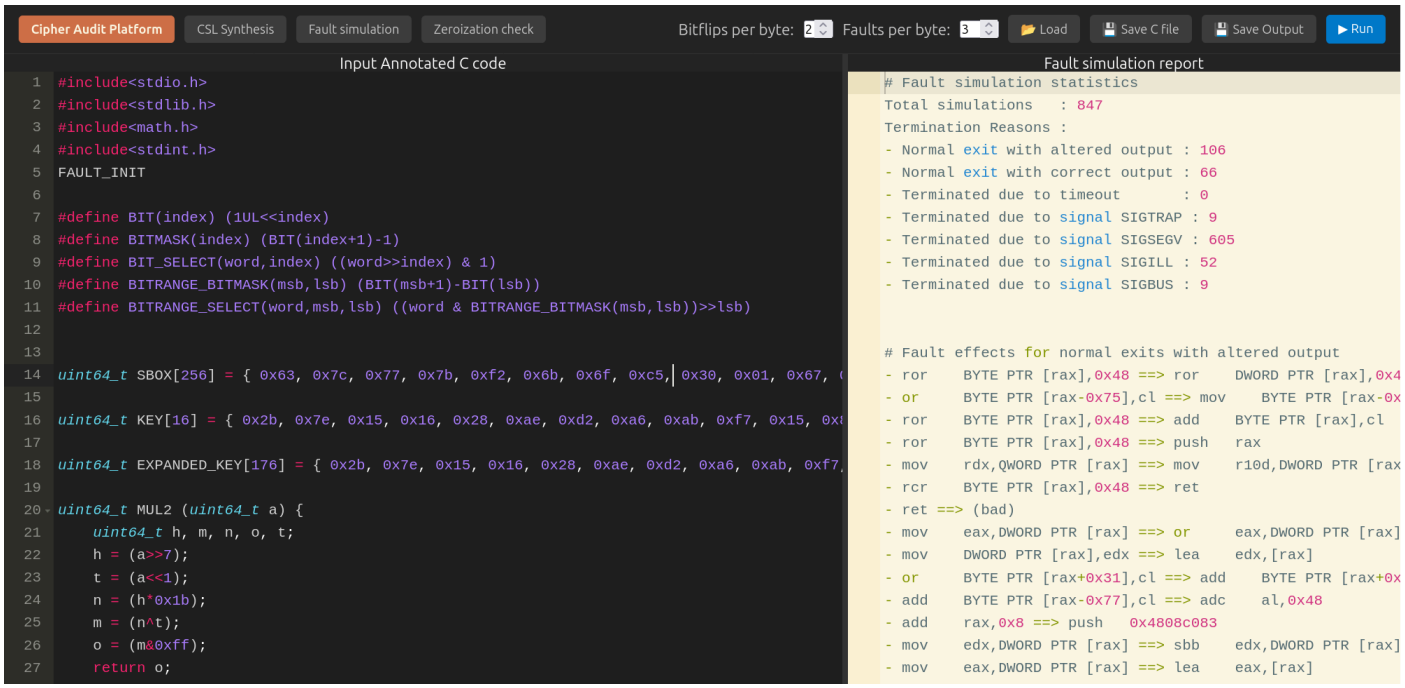
Required Annotations

Before running fault simulation, add the following annotations to the cipher program

1. *FAULT_INIT* : Defines the fault simulation helper functions in the C program
2. *FAULT_BEGIN*, *FAULT_END*
    - Marks the beginning and end of the C program range where we want to simulate faults.
    - The *FAULT_BEGIN* and *FAULT_END* should be called at the same function call level, but can be called multiple times in the program.
3. *FAULT_OUTPUT(arrayname, len)* : To mark the cipher output present in the array named arrayname which is of length len.

Web Interface

- On the web interface, the Fault Injection Simulation is present in the “Fault Simulation” tab. The left editor pane takes the annotated C code, and the fault simulation parameters (Bitflips per byte, Faults per byte) can be specified using the input fields in the header.
- On pressing the Run button, the fault simulation would be executed, and the results would be displayed on the right pane.



CLI Interface

- Run fault simulation on an annotated cipher implementation. We can specify the fault simulation parameters using the -b parameter which specifies the maximum number

of bit flips model to consider, and the `-n` parameter which specifies the number of faults per byte of the annotated region. Once completed, the tool displays the fault simulation statistics, including the list of successful faults that allowed the cipher round bytes to be corrupted and propagated to the output.

```
python3 main.py simulate-faults <annotated-c-file> -b <bitflips> -n <faults-per-byte>
```

Exercises

- Perform fault simulation on the generated C code, by adding the fault annotations to it. Examine the count of successful faults, and the instruction corruptions that enable that output, using the CAP tool.
- Vary the fault simulation parameters: Bitflips per byte and faults per byte to notice variations in the output.

04. Zeroization Verification

GNUZero is a GCC plugin that extends GCC's built-in -fanalyzer static analysis pass (GSA) . When you compile a C file with -fplugin=libscrub.so, the plugin hooks into the analysis and checks for additional vulnerability patterns- such as sensitive data left uncleared in memory (CWE-226) or uncleared stack variables (CWE-244).

```
gcc -fplugin=libscrub.so -fanalyzer -c file.c
    └── GCC analyzer pass
        └── libscrub.so hooks in → reports findings
```

The GNUZero plugin requires a custom version of GCC based on version 13.0.1, but a separate installation of GCC is not required, since it is included in the docker image, which would be downloaded by the CAP tool during the first check-zeroization run.

Overview

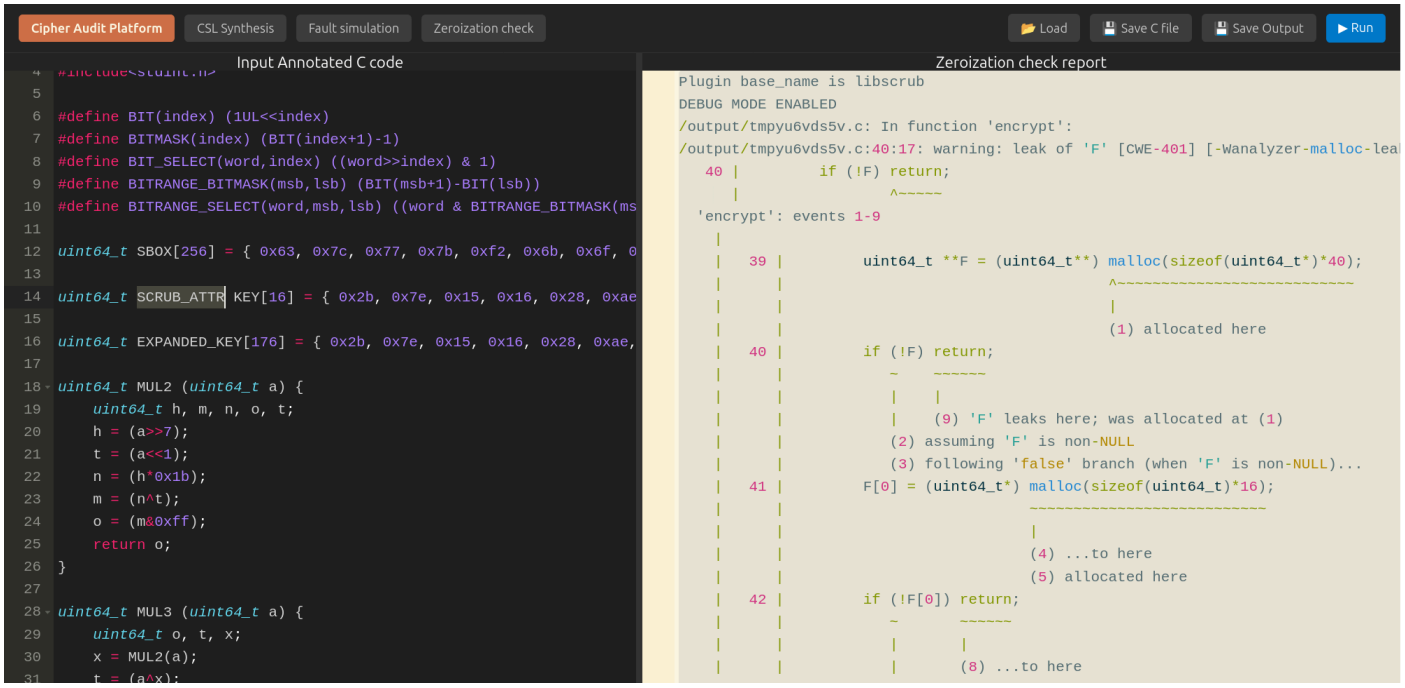
1. The tool takes as input the source code which has to be in C.
2. The verifier (who needs to have write permission to the source code) has to annotate the variables that need to be tracked as SCRUB_ATTR indicating that it is sensitive.
3. GNUZero tracks the flow of sensitive data using taint propagation. A memory object or variable annotated with SCRUB_ATTR is marked as sensitive, and the taint is propagated whenever the sensitive value is copied or used to compute new values. This enables GNUZero to track all program locations that may contain sensitive data throughout execution.
    a. For stack-allocated data, GNUZero verifies at the end of the function that all local variables annotated with SCRUB_ATTR have been securely cleared before the function returns.
    b. For heap-allocated data, GNUZero verifies that the memory is securely cleared before it is deallocated using free().
4. If there is a function that the developer or verifier knows will securely clear sensitive data when passed a pointer to it, the function can be annotated with SCRUBBER_ATTR. Once a function is annotated in this way, GNUZero assumes that any sensitive data passed to it is properly cleared and does not attempt to verify the clearing operation itself. Instead of analyzing the function's implementation, GNUZero treats the annotation as a trusted guarantee that the pointed-to data has been securely scrubbed.
5. If no function is annotated with SCRUBBER_ATTR, GNUZero tracks the state of sensitive memory throughout the function. When the function returns, or when the memory is released using free, GNUZero checks whether the memory has been overwritten with zeroes. If it determines that the memory was not properly cleared before becoming inaccessible, it raises a warning.

Note - GNUZero doesnot identify 0 assignment to a local variable as a valid erasure, instead you have to use the below pattern for zeroising local variables -

```
int local_variable = 1;
int *ptr_to_local = &local_variable;
*ptr_to_local = 0;
```

Web Interface

- On the web interface, the Zeroization verification is present in the “Zeroization check” tab. The left editor pane takes the annotated C code. When the Run button is pressed, the GNUZero zeroization check results would be displayed on the right pane.



CLI Interface

- The below command takes an annotated C file as input and checks for zeroization. During the first run, the tool will download a docker image for running GNUZero.
  `python3 main.py check-zeroization <annotated-c-file>`

Exercises

- Perform zeroization checks on the test cases shared, by annotating the key variable and running it on the CAP tool. Observe the zeroization error messages reported by GNUZero, and update the implementation to perform zeroization.
- Heartbleed demo illustrating the need for Zeroization. Run the given server program and provide the relevant client parameters to leak the key present on the stack. Add zeroization to prevent the key leak, and ensure by running the client program with the vulnerable parameters.

Example annotated C file

```c
#include "../../scrub.h"

extern SCRUBBER_ATTR void * scrubber_fn (void *);

void use_secret (int *ptr) {
    int t[4] = {0};
    if (ptr)
        t[1] = *ptr;
        // secret gets copied to t[1]
        // t[1]= 0; Zeroization commented out
}

void foo (void) {
    int SCRUB_ATTR secret[4] = { 42 };
    use_secret(secret);
    scrubber_fn (secret);
}
```

GNUZero Output

```
test/interprocedural/stack/demo.c:17:18: note: Found attribute ‘scrub’ on ‘secret’ in context of function ‘foo’
   17 |    int SCRUB_ATTR secret[4] = { 42 };
      |                   ^~~~~~
test/interprocedural/stack/demo.c: In function ‘use_secret’:
test/interprocedural/stack/demo.c:12:1: warning: leak of unscrubbed data ‘t[1]’ [CWE-226] [-Wanalyzer-malloc-leak]
   12 | }
      | ^
  ‘foo’: events 1-2
    |
    |   15 | foo (void)
    |      | ^~~
    |      | |
    |      | (1) entry to ‘foo’
    |......
    |   18 |   use_secret (secret);
    |      |   ~~~~~~~~~~~~~~~~~~~
    |      |   |
    |      |   (2) calling ‘use_secret’ from ‘foo’
    |
    +--> ‘use_secret’: events 3-7
           |
           |    6 | use_secret (int *ptr)
           |      | ^~~~~~~~~~
           |      | |
           |      | (3) entry to ‘use_secret’
           |    7 | {
           |    8 |   int t[4] = {0};
           |      |       ~
           |      |       |
           |      |       (4) region created on stack here
           |    9 |   if (ptr)
           |      |      ~
           |      |      |
           |      |      (5) following ‘true’ branch (when ‘ptr’ is non-NULL)...
           |   10 |     t[1] = *ptr;
           |      |            ~~~~
           |      |            |
           |      |            (6) ...to here
           |   11 |   //t[1] = 0; // sensitive data getting copied and cleared
           |   12 | }
           |      | ~
           |      | |
           |      | (7) ‘t[1]’ leaks here
           |
```

Explanation

Here the variable secret annotated with SCRUB_ATTR was passed to the function use_key, inside this function it got copied to ***t[1].*** We see that the ***secret*** later gets passed to the scrubber_fn which should handle clearing it off. But, since it also got copied to the variable ***t[1]*** which was not zeroed, the tool rightly raises a warning.

You will find more such examples to play with and understand the tool better in the ***test*** folder.

05. Future extensions

In the future version of the CAP tool, we plan to complete the mapping module, extend the fault simulation for data faults, and integrate the modules into the complete workflow to evaluate the resilience of cipher implementations.

Further instructions on how to use the tool can be found at this link :
https://github.com/nifey/CipherAuditPlatform



Do provide your feedback in using the tool, and any suggestion for improving the usability and functionality of the tool, in the following form:

https://forms.gle/49mwp8ByD52hCmncA



06. References

- SAFARI: Automatic Synthesis of Fault-Attack Resistant Block Cipher Implementations
  https://ieeexplore.ieee.org/abstract/document/8634921
- FortiFix: A Fault Attack Aware Compiler Framework for Crypto Implementations
  https://dl.acm.org/doi/10.1145/3650029
- FaultMeter: Quantitative Fault Attack Assessment of Block Cipher Software
  https://tches.iacr.org/index.php/TCHES/article/view/10282
- FEDS: Comprehensive Fault Attack Exploitability Detection for Software Implementations of Block Ciphers
  https://tches.iacr.org/index.php/TCHES/article/view/8552
- GnuZero: A Compiler-Based Zeroization Static Detection Tool for the Masses
  https://inria.hal.science/hal-05065294v1/document